Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int block[m], original[m];

    printf("Enter size of each block:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        original[i] = block[i]; // store original sizes
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int process[n];

    printf("Enter size of each process:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &process[i]);
```

Test Cases 2

Run Sample Cases

Run All Cases

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress 99%

Go Back

Back To Practice

Q1

Save

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Select Language : C(GCC 9.2.0)

```c
        scanf("%d", &process[i]);
    }

    // Best Fit Allocation
    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        int minFragment = 1000000;

        for (int j = 0; j < m; j++) {
            if (block[j] >= process[i]) {
                int fragment = block[j] - process[i];

                if (fragment < minFragment) {
                    minFragment = fragment;
                    bestIdx = j;
                }
            }
        }

        if (bestIdx != -1) {
            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                   i + 1, process[i], bestIdx + 1, original[bestIdx], minFragment);

            block[bestIdx] -= process[i]; // reduce available size
```

Test Cases 2

Run Sample Cases

Run All Cases

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Select Language : C (GCC 9.2.0)

```c

        for (int j = 0; j < m; j++) {
            if (block[j] >= process[i]) {
                int fragment = block[j] - process[i];

                if (fragment < minFragment) {
                    minFragment = fragment;
                    bestIdx = j;
                }
            }
        }

        if (bestIdx != -1) {
            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                   i + 1, process[i], bestIdx + 1, original[bestIdx], minFragment);

            block[bestIdx] -= process[i]; // reduce available size
        } else {
            printf("Process %d of size %d is not allocated\n", i + 1, process[i]);
        }
    }

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test

Success

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300
with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500
with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200
with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600
with Fragment 174
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
```

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300
with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500
with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200
with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600
with Fragment 174
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174
```

Test Case - 2

Success

Input :

```
3
100 200 300
4
90 150 250 400
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 90 is allocated to Block 1 of size 100 with Fragment 10
Process 2 of size 150 is allocated to Block 2 of size 200 with Fragment 50
Process 3 of size 250 is allocated to Block 3 of size 300 with Fragment 50
Process 4 of size 400 is not allocated
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 90 is allocated to Block 1 of size 100 with Fragment 10
Process 2 of size 150 is allocated to Block 2 of size 200 with Fragment 50
Process 3 of size 250 is allocated to Block 3 of size 300 with Fragment 50
```